

Метод логистической регрессии

Волгина Анастасия, Иванов Андрей
5030102/30101

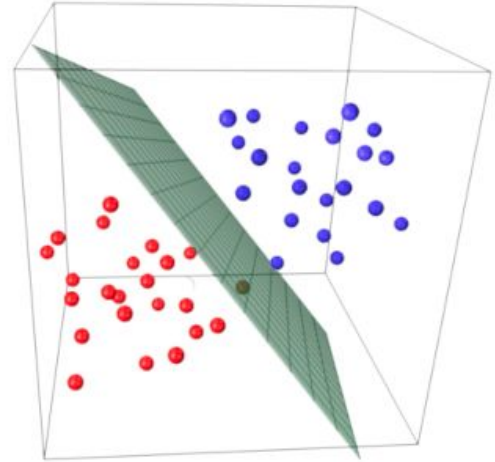
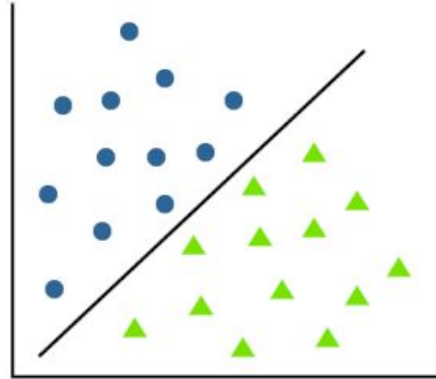
Идея: разделить пространство признаков гиперплоскостью

Постановка:

есть объекты, которые
представлены точками в
пространстве признаков

Цель:

хотим разделять объекты двух
классов с помощью линейной
функции (гиперплоскости)



Геометрия разбиения

Уравнение гиперплоскости в n -мерном пространстве:

$$w^T x + b = 0$$

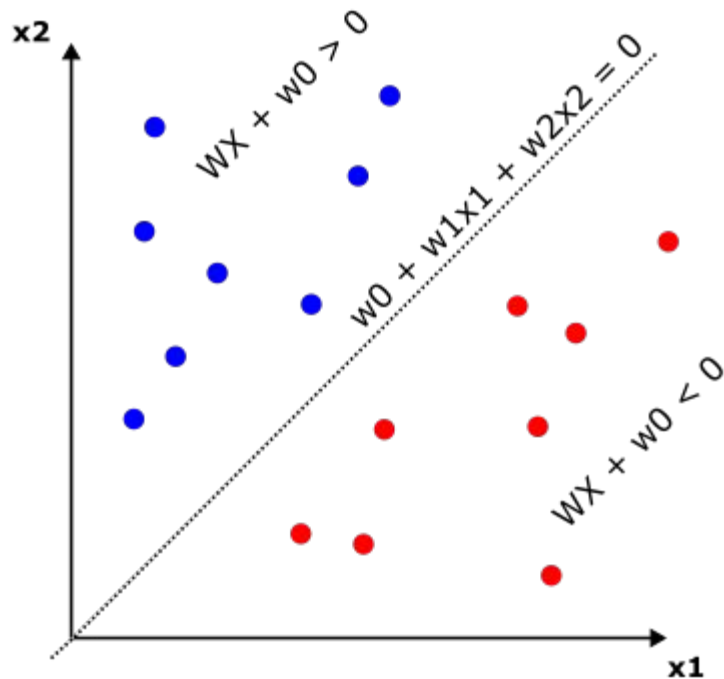
Гиперплоскость разбивает пространство на два полупространства:

$$w^T x + b > 0$$

$$w^T x + b < 0$$

Будем писать:

$$w^T x + b = z$$



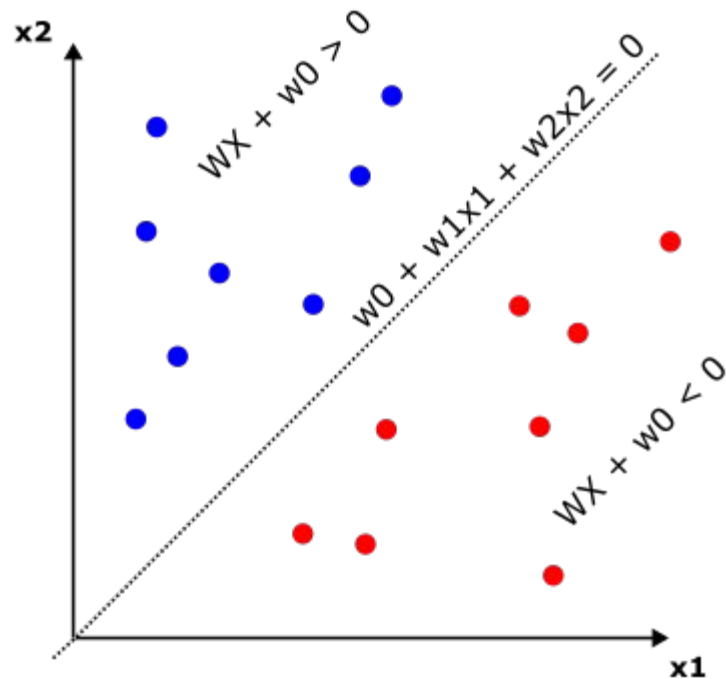
Понятие логита

Величина z – называется логит. Она отражает расстояние от объекта до гиперплоскости с учетом знака

Чем дальше объекты лежат от линии разделения, тем более мы уверены в принадлежности к соответствующему классу. А чем ближе к линии – тем менее уверены.

если z сильно положителен, то мы уверены, что +

если z сильно отрицателен, то мы уверены, что -

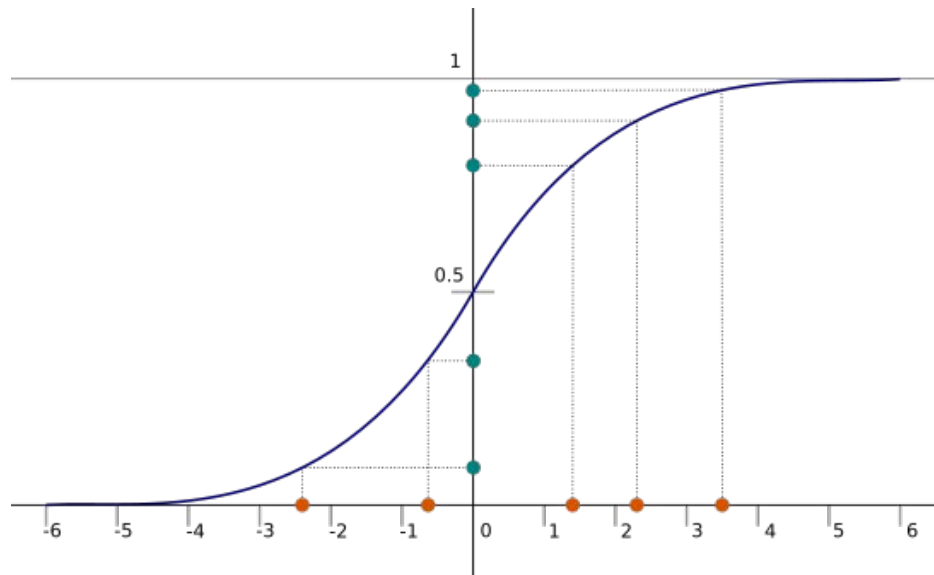


Переход от логитов к вероятностям

Значение логита находится от минус бесконечности до плюс бесконечности.

Но мы бы хотели, чтобы значения нашей “уверенности” в классе были оценками для вероятности попадания в класс.

Значит надо перевести логиты со всей числовой оси на интервал от 0 до 1. Это делается с помощью функции сигмоида.



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

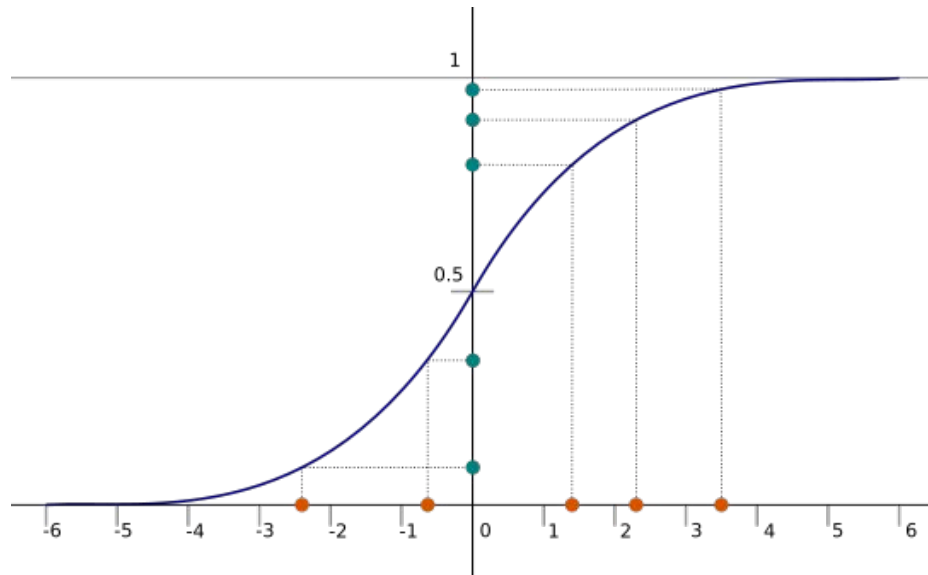
Сигмоида

Мы считаем, что оценкой уверенности отнесения объекта к классу является логит.

Но логит принимает значения на всей оси.

Значит, надо перевести логит в интервал (0, 1).

Поэтому можно считать, что значение сигмоиды от логита - оценка вероятности



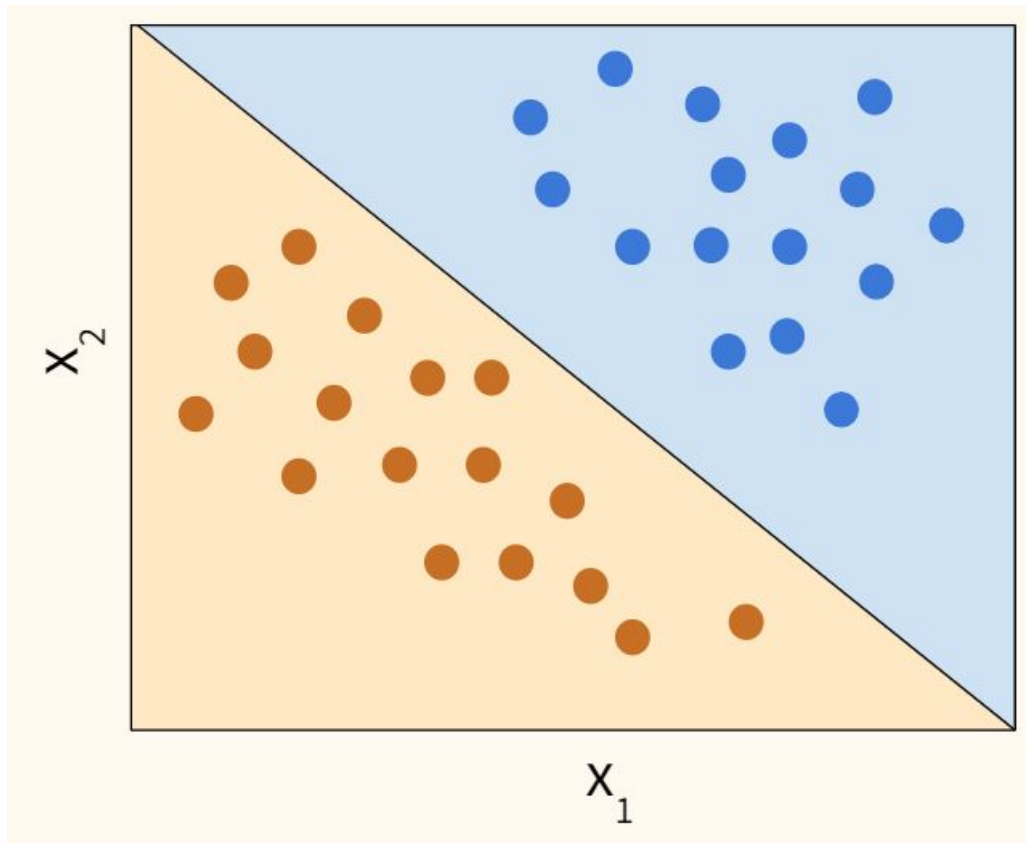
$$p = \frac{1}{1 + e^{-\langle w, x_i \rangle}}$$

Переход от классификации к оптимизации

Задача состоит в том, чтобы меняя веса w , подвинуть гиперплоскость так, что с одной стороны от линии окажется только положительный класс, а с другой только отрицательный.

Но как проверить, что все объекты оказались в нужной половине пространства?

Нужно найти **функцию потерь**



Распределение Бернулли и Функция правдоподобия

Мы знаем вероятность каждого объекта находиться в положительном классе.

То есть знаем вероятность успеха

Тогда используя распределение Бернулли можно найти правдоподобие того, что объект находится в положительном классе:

$$p_i^{y_i} (1 - p_i)^{1-y_i}$$

Будем считать, что каждый объект определяет свой класс независимо от остальных.

Тогда правдоподобие для всего набора будет выражаться так:

$$p(y | X, w) = \prod_i p(y_i | x_i, w)$$

Переход к Log Loss

Оптимизировать произведение неудобно, хочется иметь дело с суммой, поэтому логарифмируем:

$$\begin{aligned}\ell(w, X, y) &= \sum_i (y_i \log(p_i) + (1 - y_i) \log(1 - p_i)) = \\ &= \sum_i (y_i \log(\sigma(\langle w, x_i \rangle)) + (1 - y_i) \log(1 - \sigma(\langle w, x_i \rangle))).\end{aligned}$$

Переход к Log Loss

Воспользуемся свойством сигмоиды и получим:

$$\sum_i (y_i \log(\sigma(\langle w, x_i \rangle)) + (1 - y_i) \log(\sigma(-\langle w, x_i \rangle)))$$

Нас интересует w , для которого правдоподобие максимально. Чтобы получить функцию потерь, которую мы будем *минимизировать*, умножим его на минус один:

$$-\sum_i (y_i \log(\sigma(\langle w, x_i \rangle)) + (1 - y_i) \log(\sigma(-\langle w, x_i \rangle)))$$

Градиентный спуск

Нам нужно найти минимум этой функции по w .

Она дифференцируема.

Значит можно использовать градиентный спуск

Выражение градиента LogLoss'a:

$$\nabla_w L(y, X, w) = - \sum_i x_i (y_i - \sigma(\langle w, x_i \rangle))$$

Регуляризация (Борьба с переобучением)

Проблема: Без ограничений веса w могут уходить в бесконечность, для того чтобы сигмоида выдавала «чистые» 0 и 1. Это ведет к переобучению.

Решение: Добавление штрафа за размер весов прямо в Log Loss.

L2-регуляризация(Ridge):

Добавляет квадрат второй нормы w в LL. Сжимает веса к нулю, делает модель стабильной.

$$RidgeLogLoss = -\frac{1}{n} \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)) + \lambda_2 \sum_{j=1}^m w_j^2$$

L1-регуляризация(Lasso):

Добавляет первую норму w в LL.
Зануляет неважные признаки

$$LassoLogLoss = -\frac{1}{n} \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)) + \lambda_1 \sum_{j=1}^m |w_j|$$

Модификации функции потерь при дисбалансе классов

Проблема: Если в данных 99% нулей и 1% единиц, модель может просто предсказывать всем 0, и Log Loss будет низким, но модель бесполезна.

Решение 1: *Weighted Log Loss* (Взвешенный лог-лосс) — добавление весовых коэффициентов перед слагаемыми y_i и $(1-y_i)$, чтобы штраф за ошибку на редком классе был выше.

Решение 2: *Focal Loss* (динамическое масштабирование) — уменьшает вклад легко классифицируемых объектов, заставляя модель фокусироваться на сложных.

Калибровка вероятностей

Проблема: Лог рег может начать давать слишком уверенные оценки или, наоборот, смещать предсказания ближе к 0.5. Это происходит из-за того, что метрика *Log Loss* крайне чувствительна к точности самих вероятностей.

Суть решения: берем сырые выходы модели (предсказанные вероятности или логиты) и обучаем поверх них ещё одну мини-модель, которая преобразует предсказания так, чтобы они соответствовали реальной частоте классов.

Методы:

Масштабирование Платта: обучение ещё одной маленькой логистической регрессии поверх выходов основной модели. Хорошо работает на небольших выборках.

Изотоническая регрессия: находит монотонную кусочно-линейную функцию для корректировки вероятностей. Работает лучше, если данных много.

Результат: Модель не просто правильно разделяет классы, но и выдает математически точную вероятность, что напрямую минимизирует финальный *Log Loss*.

Пример

Обучающая выборка:

$$X_{train} = [[1, 5], [2, 6], [5, 7], [6, 8], [2, 5], [7, 6]]$$

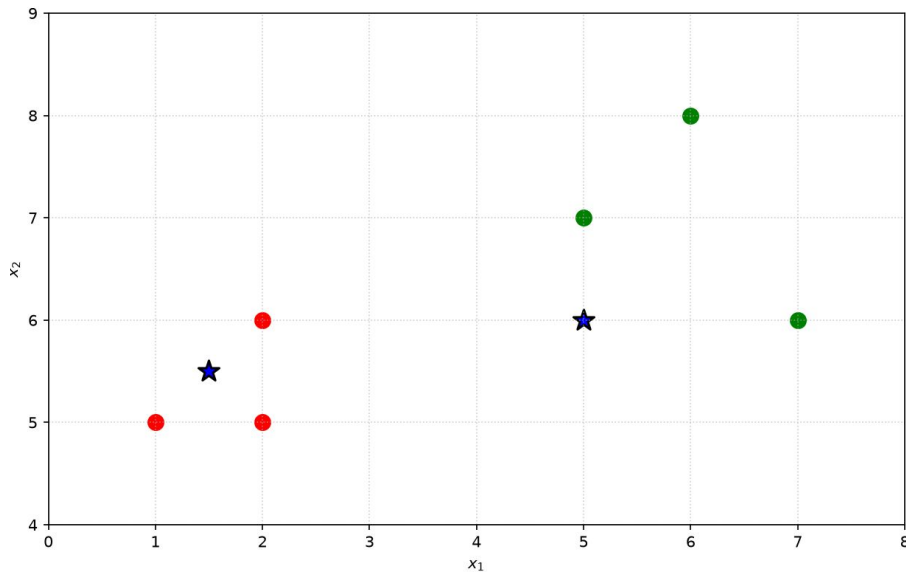
$$y_{train} = [0, 0, 1, 1, 0, 1]$$

Инициализация: $w_1 = 0, w_2 = 0, b = 0$

Градиентный спуск

Для всех объектов: $z^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)} + b = 0 \cdot x_1^{(i)} + 0 \cdot x_2^{(i)} + 0 = 0$

$$\sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}} = \frac{1}{1+e^0} = 0.5$$



Пример

Вычислим $y^{(i)} - \sigma(z^{(i)})$:

1 : 0 - 0.5 = -0.5	4 : 1 - 0.5 = 0.5
2 : 0 - 0.5 = -0.5	5 : 0 - 0.5 = -0.5
3 : 1 - 0.5 = 0.5	6 : 1 - 0.5 = 0.5

Вычислим градиент LogLoss:

$$\frac{\partial L}{\partial b} = -\frac{1}{n} \sum (y^{(i)} - \sigma(z^{(i)})) = -\frac{1}{6}(-0.5 - 0.5 + 0.5 + 0.5 - 0.5 + 0.5) = 0$$

$$\begin{aligned} \frac{\partial L}{\partial w_1} &= -\frac{1}{n} \sum x_1^{(i)} (y^{(i)} - \sigma(z^{(i)})) = -\frac{1}{6}(1 \cdot (-0.5) + 2 \cdot (-0.5) + 5 \cdot 0.5 + 6 \cdot 0.5 + \\ &+ 2 \cdot (-0.5) + 7 \cdot 0.5) = -\frac{1}{6} \cdot 6.5 \approx -1.0833 \end{aligned}$$

$$\begin{aligned} \frac{\partial L}{\partial w_2} &= -\frac{1}{n} \sum x_2^{(i)} (y^{(i)} - \sigma(z^{(i)})) = -\frac{1}{6}(5 \cdot (-0.5) + 6 \cdot (-0.5) + 7 \cdot 0.5 + 8 \cdot 0.5 + \\ &+ 5 \cdot (-0.5) + 6 \cdot 0.5) = -\frac{1}{6} \cdot 2.5 \approx -0.4167 \end{aligned}$$

Пример

Шаг обучения $\alpha = 0.1$

$$b := b - \alpha \frac{\partial L}{\partial b} = 0 - 0.1 \cdot 0 = 0$$

$$w_1 := w_1 - \alpha \frac{\partial L}{\partial w_1} = 0 - 0.1 \cdot (-1.0833) = 0.10833$$

$$w_2 := w_2 - \alpha \frac{\partial L}{\partial w_2} = 0 - 0.1 \cdot (-0.4167) = 0.04167$$

Итоговый результат градиентного спуска: $b \approx -21.41$

$$w_1 \approx 2.94$$

$$w_2 \approx 1.25$$

Пример

Тестовая выборка:

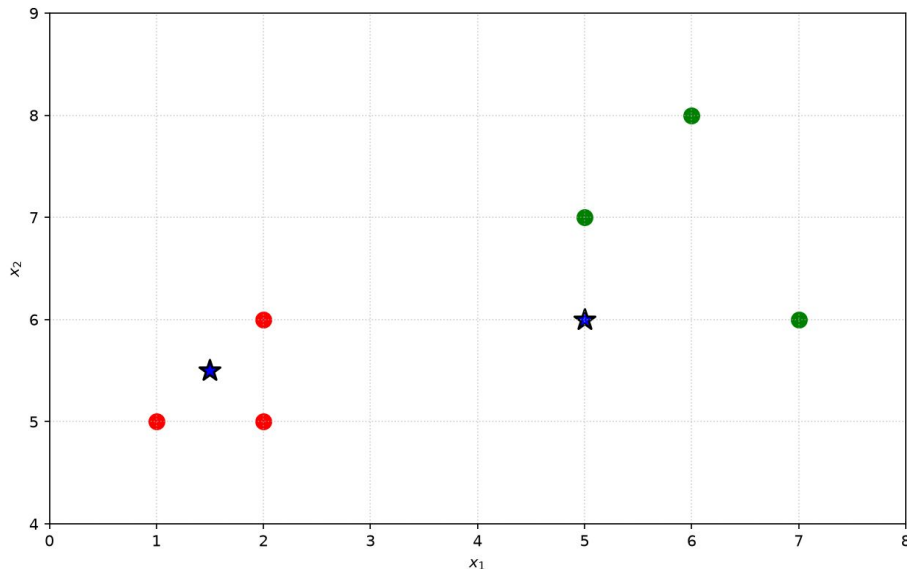
$$X_{test} = [[1.5, 5.5], [5, 6]]$$

Для первой точки:

$$z = w_1x_1 + w_2x_2 + b = 1.5 \cdot 2.94 + \\ + 5.5 \cdot 1.25 + (-21.41) = -10.125$$

$$\sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{10.125}} \approx 0.00004$$

Для второй точки аналогично получаем $\sigma(z) \approx 0.6879$



Реализация

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 1 usage
6 class LogisticRegression:
7     def __init__(self, learning_rate=0.01, epochs=1000):
8         self.learning_rate = learning_rate
9         self.epochs = epochs
10        self.weights = None
11        self.bias = None
12
13        # сигмоида
14        2 usages
15        def sigmoid(self, z):
16            return 1 / (1 + np.exp(-z))
```

Реализация

```
16      # обучение
17      1 usage
18      def fit(self, X, y):
19          num_samples, num_features = X.shape
20
21          self.weights = np.zeros(num_features)
22          self.bias = 0.0
23
24          # градиентный спуск
25          for _ in range(self.epochs):
26
27              linear_model = np.dot(X, self.weights) + self.bias
28
29              y_predicted = self._sigmoid(linear_model)
30
31              dw = (1 / num_samples) * np.dot(X.T, (y_predicted - y))
32              db = (1 / num_samples) * np.sum(y_predicted - y)
33
34              self.weights -= self.learning_rate * dw
35              self.bias -= self.learning_rate * db
```

Реализация

```
# вероятности для тестовой выборки
2 usages

def predict_probabilities(self, X):
    linear_model = np.dot(X, self.weights) + self.bias
    return self._sigmoid(linear_model)

# классы для тестовой выборки
1 usage

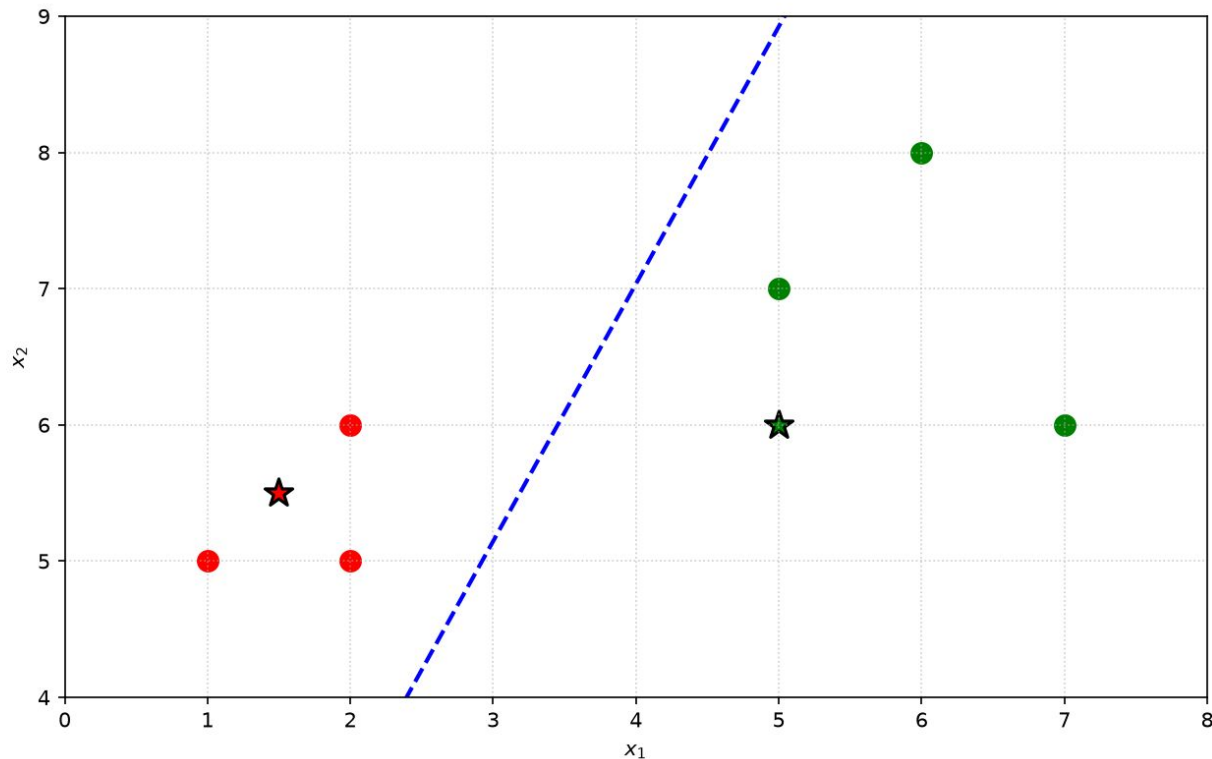
def predict(self, X, threshold=0.5):
    return np.array([1 if p >= threshold else 0 for p in self.predict_probabilities(X)])
```

Результат на примере

Результаты:

Данные: [1.5 5.5]: Вероятность "1" = 0.0002 -> Итог: 0

Данные: [5. 6.]: Вероятность "1" = 0.9996 -> Итог: 1



ИСТОЧНИКИ

<https://education.yandex.ru/handbook/ml/article/linear-models>

https://ml-cheatsheet.readthedocs.io/en/latest/logistic_regression.html

<https://scikit-learn.org/stable/modules/calibration.html>

<https://towardsdatascience.com/logistic-regression-explained-a-visual-guide-with-code-examples-for-beginners-81baf5871505/>